

TP3 - Sistemas Operativos

1. Hay 3 procesos cooperativos que acceden a 2 secciones críticas, y cada proceso tiene derecho a acceder a las 2 secciones críticas en un instante de tiempo. La estructura de los procesos es la siguiente:

P1: while (1) {	P2: while (1) {	P3: while (1) {
...	...	...
while (T&S(&lock1));	while (T&S(&lock1));	while (T&S(&lock2));
...	...	...
while (T&S(&lock2));	while (T&S(&lock2));	while (T&S(&lock1));
...	...	...
...	...	...
lock2 = 0;	lock1 = 0;	lock2 = 0;
lock1 = 0;	lock2 = 0;	lock1 = 0;
...	...	...
}	}	}

Suponga que los procesos no utilizan otro recurso aparte de los locks. Ambos locks están en cero antes que el tercer proceso comience. ¿Cual de las siguientes aseveraciones es verdadera?

Antes me gustaría aclarar que con deadlock se refiere a que dos o más procesos están esperando indefinidamente por un evento que puede ser causado por solo uno de los procesos que esperan

- a) No hay posibilidades de deadlock con esta inicialización. F
- b) Deadlock es posible y puede estar afectado un solo proceso. F
- c) Deadlock es posible y pueden estar afectados dos procesos. F
- d) Deadlock es posible y pueden estar afectados los tres procesos. V

La aseveración correcta es la última, y esto se debe a que por ejemplo arrancamos con los proceso P1 y P2 que están esperando al lock 1, el cual por estar inicializado en 0 nos indica que el recurso de la sección crítica no está siendo utilizado. Entonces al principio tanto P1 como P2 van a poder utilizarlo, el primero que se lo quede supongamos que es P1 puede volver el lock a 1 indicando que está usandolo, pero sin embargo el P2 entonces se quedaría ciclando esperando que el lock 1 se deshabilite. Ahora bien, el proceso 3 lo primero que pregunta en cambio es el lock 2, el cuál sí puede llegar a ejecutarlo pero mientras lo está usando, el segundo while de P1 no va a poder ser usado ya que lo está usando P3 al lock 2. A su vez, el segundo while del P3 tampoco ya que el que lo tiene usando es P1, por lo que se estaría produciendo un deadlock entre los dos por ese motivo, pero P2 también estaría ya que desde un principio no pudo acceder.

2. Tweedledum y Tweedledee son threads que están ejecutando sus respectivos procedimientos. El código presentado intenta que para siempre estén en intercambiando opiniones a través de una variable compartida X en estricta alternancia. Las rutinas de Sleep() y WakeUp() operan de la siguiente manera: Sleep bloquea el thread invocador. y

Wakeup desbloquea un thread específico si ese thread está bloqueado, de otra forma su comportamiento es impredecible.

```
void Tweedledum()
{
    while(1) {
        Sleep();
        x = Quarrel(x);
        Wakeup(Tweedledee thread);
    }
}

void Tweedledee()
{
    while(1) {
        x = Quarrel(x);
        Wakeup(Tweedledum thread);
        Sleep();
    }
}
```

a) Muestre un escenario en el cual el código pueda fallar.

Un escenario que puede fallar es aquel donde Tweedledee ejecute todo primero antes que siquiera Tweedledum comience, entonces dormiría a su proceso, y luego cuando el de la izquierda arranque no le queda otra que dormir, lo que llevaría a que ambos hilos estén bloqueados produciendo un deadlock entre ambos.

b) Muestre como solucionar el problema reemplazando las llamadas Sleep y Wakeup con operaciones de semáforo. No se pueden deshabilitar las interrupciones

Para solucionar este problema propongo esto:

```
sem_tweedledum = 1;
sem_tweedledee = 0;
```

```
void Tweedledum() {
    while(1) {
        sem_wait(&sem_tweedledum); // Espera su turno
        x = Quarrel(x);           // Ejecuta la acción con la variable compartida
        sem_post(&sem_tweedledee); // Despierta a Tweedledee
    }
}
```

```
void Tweedledee() {
    while(1) {
        sem_wait(&sem_tweedledee); // Espera su turno
        x = Quarrel(x);           // Ejecuta la acción con la variable compartida
        sem_post(&sem_tweedledum); // Despierta a Tweedledum
    }
}
```

De esta forma arrancaría Tweedledum siempre primero por la inicialización del semáforo, y luego de que este termine, RECIÉN ahí le manda a Tweedledee que él puede empezar a ejecutar y viceversa. De esta forma estaríamos asegurando la concurrencia entre estos dos procesos/hilos.

3. Explique la diferencia entre la espera ocupada y el bloqueo.

La espera ocupada por un lado es que mientras un proceso se encuentra en su sección crítica, cualquier otro proceso que intente ingresar a su sección crítica debe realizar un bucle continuo en la llamada a `acquire()`, es decir preguntar por una condición de si puede o no entrar. Este proceso en este ciclo estaría teniendo una espera ocupada. Este bucle continuo es claramente un problema en un sistema multiprogramación real, donde un solo núcleo de CPU se comparte entre muchos procesos. La espera ocupada también desperdicia ciclos de CPU que algún otro proceso podría usar de manera productiva.

Por otro lado, teniendo en cuenta estos desperdicios de CPU, existen los bloqueos. Su objetivo es evitar consumir recursos cuando los procesos no pueden acceder a la sección crítica. Para ello, lo que se hace es un `sleep()` a los procesos, para después despertarlo cuando salga el proceso anterior que estaba en la sección crítica generando así una buena optimización de los recursos. Un dato extra que no es una buena implementación utilizar semáforos con esta espera ocupada, ya que como se dice se pierden ciclos de CPU, para ello se los suele implementar con una cola de bloqueados en la estructura de cada semáforo.

4. Muestre que si wait y signal no son ejecutadas atómicamente, la exclusión mutua puede ser violada.

Lo primero que hay que saber es que todas las modificaciones al valor entero del semáforo en el cual justamente se utilizan las operaciones `wait()` y `signal()`, deben ejecutarse de manera atómica. Es decir, cuando un proceso modifica el valor del semáforo, ningún otro proceso puede modificar ese mismo valor simultáneamente. Además, en el caso de `wait(S)`, tanto la comprobación del valor entero de S ($S \leq 0$) como su posible modificación ($S--$) deben ejecutarse sin interrupción.

Tenemos un ejemplo básico, donde sabiendo que lo que hace `wait` es decrementar el valor de S en 1, si tuviéramos dos `wait` seguidos en el mismo programa, entonces puede ser que el primer `wait` disminuya a S , pero que no lo alcance a guardar en el registro, y justo antes de que termine el `wait` de abajo usa el mismo valor. Entonces si $S = 3$, el segundo `wait` terminaría guardando el valor 2 en S , en vez de 1 como debería ser.

5. Dé una idea de la forma en que un sistema operativo con posibilidades de desactivar las interrupciones podría implementar semáforos.

El semáforo debe ser un objeto del kernel y el espacio de usuario no debe tener permitido ningún acceso a la estructura del kernel. No queremos ningún tipo de corrupción de este tipo. Entonces el objeto semáforo debe contener un `spinlock/mutex` (para protegerlo), un contador (el propio semáforo) y una cola de espera (para procesos que intentan bajar el semáforo cuando ya está a cero).

Para bajar, el kernel adquiere un `spinlock/mutex` y también desactiva atómicamente las interrupciones cuando se realice un `wait` o un `signal`. Entonces comprueba el contador; si está por encima de 0 lo deja caer y libera el `spinlock/mutex` y ya está. De lo contrario, pone

el proceso en la cola de espera, libera el mutex/rehabilita las interrupciones, programa, y luego cuando se despierta comienza de nuevo (desde la adquisición).

Para arriba, el kernel adquiere el mutex también. Comprueba el contador si hay alguna funcionalidad de prevención de desbordamiento en el incremento (si es así, devuelve un error al espacio de usuario). Luego incrementa el contador. Si el contador fue incrementado a 1, entonces despertará al primer proceso en la cola de espera (asumiendo que haya uno). Finalmente, libera el spinlock para que el proceso despertado (u otro) pueda intentar adquirir el spinlock.

6. ¿Qué son las barreras? Brinde un ejemplo en el cual considere beneficioso su uso.

Las barreras, o también mencionadas en el libro como “memory barriers”, son utilizadas en sistemas multiprocesadores donde las instrucciones de los programas pueden ejecutarse fuera de orden, lo que puede llevar a inconsistencias en los datos. Es peor si el procesador está adoptando un **modelo de memoria débilmente ordenado** donde las modificaciones en la memoria realizadas por un procesador pueden no ser inmediatamente visibles para los demás procesadores, ya que estos optimizan el orden de ejecución de las instrucciones para mejorar el rendimiento.

Esto significa entonces que, en procesadores con un modelo de memoria débilmente ordenado, un hilo en un procesador podría no ver inmediatamente los cambios realizados por otro hilo en otro procesador. Justamente para eso existen las barreras como dice el inciso, y un ejemplo beneficioso es usarlo en programas de lectura/escritura donde las memory barrier (las cuáles son instrucciones) aseguran que las operaciones de carga (lectura) y almacenamiento (escritura) de memoria se completen antes de realizar nuevas operaciones. Es decir, las barreras de memoria imponen un orden de ejecución garantizado ya que al ejecutarse una, no pasa a la otra operación hasta que se haya terminado por su decisión se haga visible para las demás. Es muy buena para el algoritmo de Peterson y ayudar a la exclusión mutua evitando el reordenamiento de instrucciones que podría violar la exclusión mutua. Sin embargo, estas tienen mejor uso para operaciones de bajo nivel.

Ejemplo:

```
while (!flag);  
    memory_barrier();  
print x;
```

Si agregamos aquí una barrera de memoria, garantiza que el valor de flag se cargue (lea) antes de acceder al valor de x, evitando que el código se ejecute en un orden incorrecto.

```
x = 100;  
memory_barrier();  
flag = true;
```

La barrera de memoria asegura que la asignación de x se complete antes de establecer flag=true. Sin la barrera, el procesador podría reordenar estas instrucciones, lo que podría

hacer que flag se establezca en true antes de que x haya sido asignado a 100, causando un comportamiento incorrecto.

7. ¿Pueden dos hilos (threads) en el mismo proceso sincronizarse mediante un semáforo de kernel, si los hilos son implementados en el kernel? ¿Qué pasa si se implementan en espacio de usuario? Suponga que ningún hilo de otro proceso tiene acceso al semáforo. Analice sus respuestas.

En primer lugar, debemos entender qué es un semáforo de kernel. Un semáforo de kernel es un mecanismo de sincronización proporcionado por el kernel del sistema operativo para permitir que varios procesos o subprocesos accedan a un recurso compartido de forma mutuamente excluyente. Funciona manteniendo un recuento de la cantidad de recursos disponibles y bloqueando cualquier proceso o subproceso que intente acceder al recurso cuando el recuento sea cero.

Ahora bien, si el kernel implementa dos subprocesos en el mismo proceso mediante hilos, pueden sincronizarse utilizando un semáforo de kernel porque tienen acceso a los mismos recursos del kernel. El semáforo de kernel se puede utilizar para proteger un recurso compartido y los subprocesos pueden utilizar el semáforo para coordinar su acceso al recurso. Distinto es cuando usamos kernel de librería como son los kernel de espacio de usuario, donde con estos no podemos saber en qué estado se encuentra el sistema por lo que no podríamos controlar o decirle al kernel cuando puede dar las señales clásica de los semáforos.

8. ¿Cuáles son las diferencias entre un semáforo binario y un mutex?

Para resolver problemas de sección crítica se encuentran los mutex, los cuales son herramientas del software de alto nivel que controlan la condición de carrera. Como su nombre lo dice, garantizan exclusión mutua para una pieza de código en donde los coloquemos. Por otro lado, los semáforos binarios que pueden tener valores de 0 a 1 tienen un comportamiento similar a los mutex locks, y son capaces de generar exclusión mutua, pero estos tienen un enfoque más para sincronizar dos procesos distintos por ejemplo.

En base a lo que dice el Ing. Piumatti, la implementación del mutex (al menos, la dada en tannenbaum) es una muy cercana al hardware (en assembler), lo cual hace que sea muy eficiente, además de que en general es fácil de implementar. A su vez, el proceso que bloquea el mutex (pone el valor en 0) debe ser el mismo que lo libera, mientras que en el semáforo binario podría no pasar esto.

9. Dado el clásico problema de productores y consumidores, se plantea una solución utilizando semáforos con el siguiente código:

<pre> Typedef int semaforo; . typedef char* msg; int N=100; /*Longitud del buffer */ semaforo mutex = 1; /*Da la exclusión mutua */ semaforo lleno = 0; /*Cuenta lugares llenos */ semaforo vacio = N; /*Cuenta lugares vacíos */ </pre>	<pre> Productor() { msg mensaje; while(TRUE) { producir(mensaje); down(&mutex); down(&vacio); entrar_msg(mensaje); up(&mutex); up(&lleno); } } </pre>	<pre> Consumidor() { msg mensaje; while(TRUE) { down(&lleno); down(&mutex); remover_msg(mensaje); up(&mutex); up(&vacio); consumir_msg(mensaje); } } </pre>
--	---	---

¿La solución planteada es válida?. En caso de que no lo sea, explique por qué.

La solución que se plantea arriba respecto al clásico problema de productor/consumidor no es correcta. La diferencia que se puede apreciar, es que primero habría que decrementar el contador vacío del buffer en vez de meternos dentro del mutex y hacerlo allí. ¿Por qué esto sería un problema? Si tenemos en cuenta que al entrar en una porción de código “encerrada” en un mutex estamos generando la exclusión mutua, y por ende controlando la condición de carrera de que varios procesos acceden y manejan datos compartidos concurrentemente, no pasaría. Si dentro del mismo entonces hacemos wait(&vacio), dejamos justamente que se accedan a datos compartidos concurrentemente, violando esta condición de carrera. Por eso lo mejor es primero realizar el wait de vacío para indicar que se produjo un ítem, y luego mediante el la exclusión mutua del mutex colocar el nuevo ítem en el buffer.

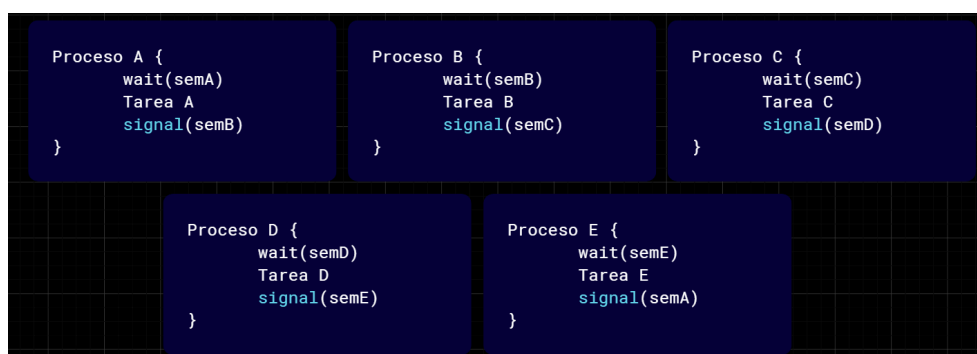
10. Para las siguientes secuencias de ejecución: Considere que un proceso produce un elemento, los elementos son A, B, C, D y E. Por lo tanto, en cada una de las secuencias hay 5 procesos. Realizar la sincronización de los procesos utilizando semáforos para cada uno de los casos especificados. Recuerde hacer un uso eficiente de los recursos.

a) La secuencia permitida es: ABCDEABCDEABCDEABCDEABCDE...

Inicialización:

semA = 1;
semB = semC = semD = semE = 0

Programa:



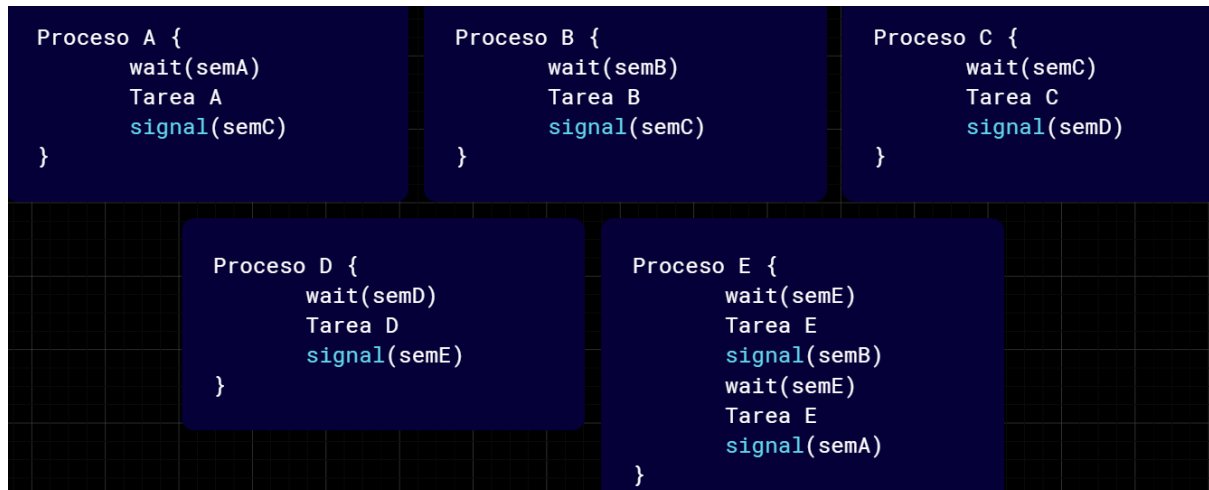
b) La secuencia permitida es: ACDEBCDEACDEBCDEACDEBCDEACDEBCDE...

Inicialización:

semA = 1;

semB = semC = semD = semE = 0

Programa:



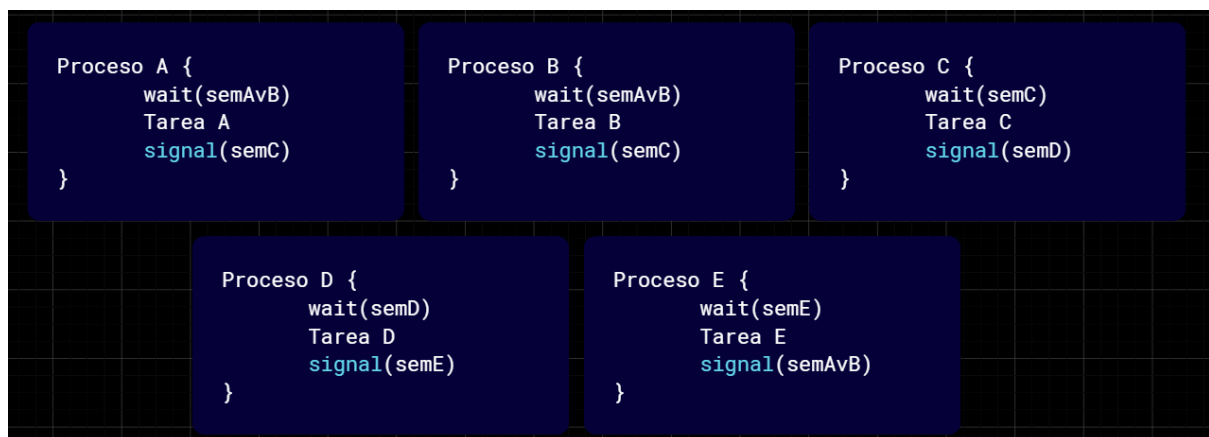
c) Las secuencia permitida es: (AvB)CDE(AvB)CDE(AvB)CDE(AvB)CDE ...

Inicialización:

semAvB = 1;

semC = semD = semE = 0

Programa:



d) La secuencia permitida es: (AvB)CE(AvB)(AvB)DE(AvB)CE(AvB)(AvB)DE...

Inicialización:

semAvB = 1;

semC = semD = semE = 0

Programa:

Hecho en C.

11. Analice la siguiente solución para el problema de los filósofos. ¿La solución propuesta tiene algún problema? De ser así explique cual es y cómo lo solucionaría.

```
#define N 6
#define Izquierdo (i-1) % N
#define Derecho (i+1) % N
#define Pensando 0
#define Tiene_Hambre 1
#define Comiendo 2
typedef int semaforo

int estado[N];
semaforo mutex = 1;
semaforo s[N];

void filosofo (int i)
{
    while (TRUE) {
        pensar();
        tomar_tenedores(i);
        comer();
        dejar_tenedores(i);
    }
}

void tomar_tenedores (int i)
{
    down(&mutex);
    estado[i] = Tiene_Hambre;
    test(i);
    up(&mutex);
    down(&s[i]);
}

void dejar_tenedores (int i)
{
    down(&mutex);
    estado[i] = Pensando;
    up(&mutex);
    test(Izquierdo);
    test(Derecho);
}

void test (int i)
{
    if (estado[i] == Tiene_Hambre && estado[Izquierdo] != Comiendo && estado[Derecho] != Comiendo) {
        estado[i] = Comiendo;
        up(&s[i]);
    }
}
```

Por empezar, la cantidad de filósofos es 5, por lo que $N = 6$ estaría mal por más que se pueda representar de esa forma que no me fijé ya que al ver el código no mapeas el problema correctamente. Luego la problemática real es que cuando en `dejar_tenedores()` termina la exclusión mutua con el `up(&mutex)` y sale de la sección crítica antes de testear si los vecinos pueden comer. Esto está mal ya que ese mutex debería estar al final de la función para garantizar que cuando el filósofo tenía los dos tenedores, esos palitos eran justamente los que necesitaban el de la izquierda y el de la derecha. Porque podría pasar que luego del mutex no se testee en F2 los dos `test()` para F1 y F3, y podría decirte que F2 que aún tiene los tenedores, se encuentra al lado de F1 y F3 que también están comiendo, algo imposible para las restricciones del problema.

12. En la fábrica de bicicletas Mountain Bike, tenemos tres operarios que denominaremos OP1, OP2 y OP3. OP1 monta ruedas, OP2 monta el cuadro de las bicicletas, y OP3, el manillar. Un cuarto operario, el Montador, se encarga de tomar dos ruedas, un cuadro y un

manillar, y arma la bicicleta. Sincronizar las acciones de los tres operarios y el Montador utilizando semáforos, en los siguientes casos:

Restricción: el OP1 solo produce de a una rueda por vez.

- a) Los operarios no tienen ningún espacio para almacenar los componentes producidos pero si tiene espacio para fabricar de a uno, y el Montador no podría tomar ninguna pieza si esta no ha sido fabricada previamente por el correspondiente operario.



- b) Los operarios no tienen ningún espacio para almacenar los componentes producidos pero si tiene espacio para fabricar de a uno, y en el caso del OP1 tiene espacio para fabricar las dos ruedas y el Montador no podría tomar ninguna pieza si esta no ha sido fabricada previamente por el correspondiente operario.



13. El problema de los fumadores de cigarrillos (Patil 1971). Considere un sistema con tres procesos fumadores y un proceso agente. Cada fumador está continuamente armando y fumando cigarrillos. Sin embargo, para armar un cigarrillo, el fumador necesita tres ingredientes: tabaco, papel y fósforos. Uno de los procesos fumadores tiene papel, otro tiene el tabaco y el tercero los fósforos. El agente tiene una cantidad infinita de los tres materiales. El agente coloca dos de los ingredientes sobre la mesa. El fumador que tiene el ingrediente restante armaría un cigarrillo y se lo fumaría, avisando al agente cuando termina. Entonces, el agente coloca dos de los tres ingredientes y se repite el ciclo. Escriba un programa para sincronizar al agente y los fumadores utilizando semáforos.



14. El problema del barbero dormilón (Dijkstra 1971). Una barbería se compone de una sala de espera con n sillas y la sala de barbería donde se encuentra la silla del barbero. Si no hay clientes a quienes atender, el barbero se pone a dormir. Si un cliente entra y todas las sillas están ocupadas, el cliente sale de la barbería. Si el barbero está ocupado, pero hay sillas disponibles, el cliente se sienta en una. Si el barbero está dormido, el cliente lo

despierta. Escriba un programa para coordinar al barbero y sus clientes utilizando semáforos.

```
semSillas = N;
semClientes = 0;
semBarbero = 0;
mutexSillas = 1;

Cliente {
    while(1) {
        wait(mutexSillas)
        if(trywait(semSillas) == 0) {
            wait(semSillas) // llega cliente y se sienta
            signal(semClientes) // se avisa al barbero
            /*SC*/ signal(mutexSillas)
            wait(semBarbero) // espera a que lo llame el barbero
            ir_a_cortarse()
        }
        else {
            signal(mutexSillas)
            salir_barbería()
        }
    }
}

Barbero {
    while(1) {
        wait(semClientes) // espera a que llegue un cliente durmiendo
        wait(mutexSillas)
        signal(semSillas) // se desocupa una silla
        signal(semBarbero) // se manda cliente a cortar
        cortar_pelo()
        signal(mutexSillas)
    }
}
```

15. Escriba un algoritmo con semáforos, que controle el acceso a un archivo, de tal manera que se permita el acceso en lectura a varios procesos o a un solo escritor en forma exclusiva.

Es básicamente el problema del lector/escritor. Para él tenemos varios tipos de soluciones, podemos darle prioridades a los lectores, a los escritores o manejar una equitatividad.

Hecho en C.

18. El oso y las abejas. Se tienen n abejas y un oso hambriento. Comparten un tarro de miel. Inicialmente el tarro de miel está vacío, su capacidad es M porciones de miel. El oso duerme hasta que el tarro de miel se llene, entonces se come toda la miel y vuelve a dormir. Cada abeja produce una porción de miel que coloca en el tarro, la abeja que llena el tarro de miel despierta al oso. Escriba un programa para que sincronice a las abejas y al oso.

Hecho en C.

19. Los servidores pueden diseñarse de modo que limiten el número de conexiones abiertas. Por ejemplo, un servidor puede autorizar solo N conexiones simultáneas de tipo socket. En cuanto se establezcan las N conexiones, el servidor ya no aceptaría otra conexión entrante hasta que una conexión existente se libere. Explique cómo puede utilizar un servidor los semáforos para limitar el número de conexiones concurrentes.

Un servidor podría tener un semáforo contador el cual esté inicializado en la cantidad N de conexiones que puede hacer. Entonces habría un `sem_trywait(conexiones)`, el cual te permitiría indicar justamente que se van realizando conexiones hasta que el semáforo llegue a 0. Si este llegara a 0, entonces se le avisaría al cliente que desea conectarse que no hay lugar y que intente en otro momento. Podría venir alguien estando todo ocupado ya que esta función es no bloqueante.

Otra opción sería poner sólo `wait`, pero no te daría esta libertad. En ambas situaciones una vez que se termine de usar una conexión habría que hacer señal de la misma indicando que ya está disponible.

20. Suponga tener n usuarios que comparten 2 impresoras. Antes de utilizar la impresora, el $U[i]$ invoca `requerir(impresora)`. Esta operación espera hasta que una de las impresoras esté disponible, retornando la identidad de la impresora libre. Después de utilizar la impresora, $U[i]$ invoca a `liberar(impresora)`.

a) Resuelva este problema considerando que cada usuario es un thread y la sincronización se realiza utilizando semáforos.

```

sem_t impresora1, impresora2;

void* usuario[i](void* arg) {
    requerir(impresora, i);
    SC printf("Usuario %d utilizando la impresora.", i);
    liberar(impresora);
}

void* requerir(impresora[i]) {
    wait(impresora1 | impresora2)
    (if try_wait(p1) == 0)
        *impresora = p1;
    (if try_wait(p2) == 0)
        *impresora = p2;
}

void* liberar(impresora){
    signal(*impresora)
}

```

b) Resuelve este problema considerando que cada usuario tiene una prioridad única. Se otorga la impresora al usuario con mayor prioridad que está esperando.

Tendría que contar con alguna estructura de datos que controle los temas de prioridad. Porque si tengo más de 2 usuarios no sabría como hacerlo.

21. Cómo se modificaría el ejercicio 2 de la sección problemas del Trabajo Práctico Nro. 3 (diagrama de transición de procesos) si se previera el uso de semáforos para sincronizar los procesos entre sí. Que nuevos estados habría que agregar al diagrama de transiciones y qué nuevas rutinas de manejo de las mismas habría que prever si:

- a) Se incluye un semáforo que controla el uso del canal correspondiente a las unidades de cinta y otro al correspondiente a las unidades de disco.
- b) Se incluye un semáforo que controla la obtención dinámica de espacio, en memoria central por parte de los procesos en ejecución.